Data Structures & Algorithms (CSC-214)

Lecture 5: Inserting into Sorted Arrays

STORE HIGH SCORES FROM A GAME USING ARRAYS

Only highest scores are retained

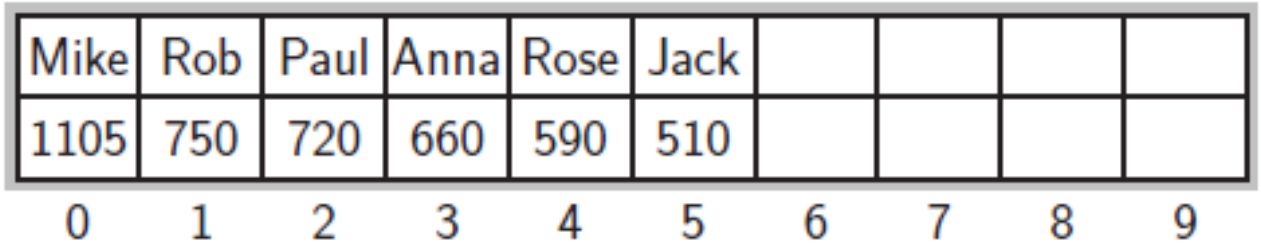

Figure 3.1: The *entries* array of length eight storing six GameEntry objects in the cells from index 0 to 5. Here *maxEntries* is 10 and *numEntries* is 6.

```
class GameEntry {                                   // a game score entry
public:
  GameEntry(const string& n="", int s=0); // constructor
  string getName() const;                           // get player name
  int getScore() const;                             // get score
private:
  string name;                                      // player's name
  int score;                                        // player's score
};
```

Code Fragment 3.1: A C++ class representing a game entry.

```
class Scores {                                  // stores game high scores
public:
  Scores(int maxEnt = 10);                      // constructor
  ~Scores();                                    // destructor
  void add(const GameEntry& e);                 // add a game entry
  GameEntry remove(int i)                       // remove the ith entry
      throw(IndexOutOfBounds);
private:
  int maxEntries;                               // maximum number of entries
  int numEntries;                               // actual number of entries
  GameEntry* entries;                           // array of game entries
};
```

Code Fragment 3.3: A C++ class for storing high game scores.

```
Scores::Scores(int maxEnt) {                    // constructor
  maxEntries = maxEnt;                          // save the max size
  entries = new GameEntry[maxEntries];          // allocate array storage
  numEntries = 0;                               // initially no elements
}

Scores::~Scores() {                             // destructor
  delete[] entries;
}
```

Code Fragment 3.4: A C++ class GameEntry representing a game entry.

```
void Scores::add(const GameEntry& e) {    // add a game entry
  int newScore = e.getScore();            // score to add
  if (numEntries == maxEntries) {         // the array is full
    if (newScore <= entries[maxEntries-1].getScore())
      return;                             // not high enough - ignore
  }
  else numEntries++;                      // if not full, one more entry

  int i = numEntries-2;                   // start with the next to last
  while ( i >= 0 && newScore > entries[i].getScore() ) {
    entries[i+1] = entries[i];            // shift right if smaller
    i--;
  }
  entries[i+1] = e;                       // put e in the empty spot
}
```

Code Fragment 3.5: C++ code for inserting a GameEntry object.

```
GameEntry Scores::remove(int i) throw(IndexOutOfBounds) {
  if ((i < 0) || (i >= numEntries))          // invalid index
    throw IndexOutOfBounds("Invalid index");
  GameEntry e = entries[i];                  // save the removed object
  for (int j = i+1; j < numEntries; j++)
    entries[j-1] = entries[j];               // shift entries left
  numEntries--;                              // one fewer entry
  return e;                                  // return the removed object
}
```

Code Fragment 3.6: C++ code for performing the remove operation.

Lecture content adapted from Michael T. Goodrich textbook, chapters 3.